

Project 4: Distributed Key-Value Storage System

Detailed stage-wise build plan for a placement-grade C++ distributed systems project

Target outcome: Build a technically credible distributed systems project that demonstrates C++, Linux, networking, concurrency, persistence, sharding, replication, fault tolerance and performance engineering. The final resume version should contain measured throughput/latency/scalability results - never invented numbers.

1. Project Scope

Project: Distributed Key-Value Storage System

Recommended stack: C++17/20, Linux, POSIX sockets, CMake, threads, mutex/shared_mutex, GoogleTest or Catch2, Docker, Python for benchmarking and plotting.

Core user story: A client can PUT, GET and DELETE key-value pairs through a network protocol. The system persists data, handles concurrent clients, scales across multiple nodes, replicates data and recovers from simulated failures.

What makes it resume-worthy

- Do not stop at a single-process hash table. The differentiating value comes from concurrency, persistence, distribution, replication and benchmarking.
- Build each layer incrementally so every stage produces a runnable system and a measurable baseline.
- Use the project to demonstrate concepts commonly tested in SDE interviews: sockets, TCP, threads, synchronization, data structures, storage, consistency, failure handling and performance.

2. Target Architecture

Client -> TCP connection -> request parser -> request router -> storage engine -> persistence layer.

In the distributed phase, add a cluster/router layer, consistent-hashing ring, replica placement and node health/failure handling.

Component	Purpose	Suggested implementation
Client	Send commands and measure latency	C++ CLI client or Python benchmark client
Protocol	Encode requests/responses	Simple text or length-prefixed binary protocol
Server	Accept and dispatch connections	POSIX sockets + thread pool
Storage engine	Fast in-memory lookup	std::unordered_map initially
Persistence	Recover state after restart	WAL + periodic snapshots
Cluster layer	Route keys to nodes	Consistent hashing
Replication	Maintain copies	Primary/replica model
Observability	Measure behavior	Structured logs + metrics

3. Stage-Wise Build Plan

Stage 0 - Design and Environment (0.5-1 day)

- Set up a C++17/20 project with CMake, compiler warnings, formatting and a unit-test framework.
- Create modules for networking, protocol, storage, persistence, cluster membership and benchmarking instead of one monolithic source file.
- Write a short design document defining commands, error codes, thread model, data ownership and the initial consistency model.

Stage 1 - Single-Node Key-Value Engine (2-3 days)

- Implement PUT, GET and DELETE against an in-memory hash table.
- Define clear success/error responses and key/value limits.
- Add unit tests for missing keys, overwrites, deletes, empty values and invalid requests.
- Create a CLI client so the system is demonstrable end-to-end.

Exit criteria: A clean client-server demo with automated tests and a reproducible command sequence.

Stage 2 - TCP Networking and Request Handling (2-3 days)

- Implement TCP accept/read/write loops and a simple framing strategy so partial reads do not corrupt requests.
- Add request parsing, validation and response serialization.
- Handle disconnects, malformed packets and server-side errors cleanly.
- Add integration tests that launch a real server and exercise it through sockets.

Stage 3 - Concurrency and Thread Pool (3-4 days)

- Introduce a bounded worker thread pool instead of creating an unbounded thread per connection.
- Protect shared state with appropriate mutexes/read-write locks; minimize lock scope.
- Build a concurrent benchmark with configurable client counts.
- Record throughput, p50/p95/p99 latency, CPU utilization and memory use.

Benchmark checkpoint: Establish the single-threaded baseline first. Every later optimization should be compared against this baseline. Keep raw benchmark outputs under version control so resume metrics remain reproducible.

Stage 4 - Persistence and Recovery (3-4 days)

- Implement a write-ahead log so successful mutations can be replayed after restart.
- Add periodic snapshots/checkpoints to avoid replaying an unbounded log.
- Define crash/restart tests: kill the process after batches of writes, restart it and verify recovered state.
- Measure write overhead introduced by persistence.

Stage 5 - Multi-Node Cluster and Consistent Hashing (4-5 days)

- Run multiple storage processes locally, preferably with Docker Compose.
- Implement a consistent-hashing ring to map keys to nodes.
- Add node join/leave behavior and verify that only the expected key ranges move.
- Benchmark scaling from 1 to 2 to 3+ nodes.

Stage 6 - Replication and Failure Handling (3-4 days)

- Add configurable replication factor, e.g. 2 or 3.
- Define primary/replica responsibilities and replication acknowledgement behavior.
- Simulate node crashes and verify that reads/writes continue according to the chosen availability/consistency trade-off.
- Document limitations honestly; do not claim consensus or strong consistency unless you actually implement it.

Stage 7 - Performance Engineering (2-3 days)

- Profile CPU hotspots, lock contention, network overhead and serialization costs.
- Compare alternative synchronization strategies where meaningful.
- Tune batch sizes, connection reuse, worker counts and storage operations.
- Run repeatable benchmark matrices across client concurrency and cluster size.

Stage 8 - Hardening, Testing and Demo (2-3 days)

- Add unit, integration, concurrency, recovery and failure-injection tests.
- Create a Docker Compose demo for a 3-node cluster.
- Write a README with architecture diagram, protocol examples, design trade-offs and benchmark methodology.
- Capture final metrics for the resume: requests/sec, p50/p95/p99 latency, scaling factor, recovery time and resource usage.

4. Metrics to Collect for the Resume

Metric	How to measure	Useful resume consequence
Throughput	Requests/sec under fixed workload	Sustained X K req/s
p99 latency	Latency distribution under concurrency	Y ms p99
Scaling	1 vs N nodes/clients	Xx throughput scaling
CPU	Process/system monitoring	Y% lower CPU
Memory	RSS/peak memory	Y MB or Z% reduction
Recovery	Crash + restart benchmark	Recovered X keys in Y sec
Failure handling	Kill a node during workload	Maintained X% request success

5. Suggested Repository Structure

```
kv-store/  
CMakeLists.txt  
include/ networking/ storage/ persistence/ cluster/  
src/ networking/ storage/ persistence/ cluster/  
tests/ unit/ integration/ failure/  
benchmarks/  
docker/  
docs/architecture.md  
README.md
```

6. Interview Topics You Must Be Able to Explain

- TCP connection lifecycle, partial reads/writes and request framing.
- Why a thread pool is preferable to one thread per request at scale.
- Mutex vs shared_mutex, lock contention and race conditions.
- Hash tables, consistent hashing and why rebalancing occurs.
- Write-ahead logging, crash recovery and durability.
- Replication, consistency vs availability and what your system does during node failure.
- Benchmark methodology and why p99 latency is often more informative than average latency.
- Which parts are production-grade and which are intentionally simplified for a learning project.

7. Final Resume Target

Ideal final project title: Distributed Key-Value Storage System | C++, Linux, TCP/IP, Multithreading, Distributed Systems, Docker

Target bullets: 2-3 bullets using the ABC format: Action verb + technical context + measured consequence. Example structure: *Engineered a distributed key-value store in C++ supporting concurrent TCP clients and replication across X nodes, sustaining YK requests/sec at Z ms p99 latency.*

Do not fabricate metrics. X/Y/Z are placeholders until the benchmark suite produces real measurements. Keep benchmark scripts and raw results so every number on the resume can be defended in an interview.